MongoDB

ADVANCE DBA COURSE

by:

Mohammed Abdul Sami

Table of content

MongoDB 1
- Introduction 5
- About Trainer 5
- Introduction 6
  - NoSQL Database 7
    - Evolution of NoSQL 7
    - NoSQL database Classification 7
    - NoSQL classification and Adherence by Vendors 8
    - NoSQL vs RDBMS 8
  - CAP Theorem 8
    - Lets Understand CAP Theorem 8
    - PACELC theorem 9
  - Markup Languages 9
    - What is JSON? 9
    - JSON Data Structures 9
    - BSON 10
    - What is YAML? 11
  - Introduction to MongoDB 11
    - History 11
    - Features of MongoDB 12
    - MongoDB Architecture 12
    - Object Hierarchy 13
    - MONGO DB Vs Traditional RDBMS 13
    - Data Types 14
    - Storage 15
  - MongoDB Data Models 18
    - Data model design 18
    - Data Model Relationships 19
- Installation 24
  - MongoDB Editions 25
    - MongoDB Community Server 25
    - MongoDB Enterprise Server 25
    - MongoDB Atlas 25
  - Supported platforms 25
  - Installing Community Edition 26

Pre Requisites 26
Installing on RHEL using RPMs 26
Installing on RHEL using tgz 26
Managing MongoDB Services 27
Connecting to MongoDB server using mongo client 28
Database and Collections 28
System Databases 29
Database Commands 29
Collections 30
CRUD Operations 31
MongoDB Operators 32
Comparison Operators: 32
Logical Operators: 32
Element Operators 33
The Crud Operations 33
insert Method 33
find Method 35
Updating Data 37
Deleting Data 40
Advance Collection Creation 40
Creating capped collection 40
Collection Validations 41
Aggregation 44
Aggregation stages 45
$match stage 45
$group Stage 45
$bucket stage 47
$bucketAuto stage 47
$addFields stage 48
$project stage 48
$lookup stage 49
MongoDB Security and User Administration 50
Authentication Mechanism 51
challenge-response 51
SCRAM 51
Authentication in Enterprise version of MongoDB 51

For internal use only not for redistribution or sale

Role-Based Access Control 52
Resource 52
Privileges 52
Roles 52
Inherited Privileges 52
Viewing Roles 52
Privileges (Actions) 53
Built-In Roles 54
User-Defined Roles 56
Enabling Security 60
Localhost Exception 60
Creating Super User 60
User Management 60
Create database administrator: 61
Connecting To MongoDB Using User 61
Granting Roles to User 61
Revoking Roles from User 61
Changing User Password 61
Authenticate to Another Database within Shell 62
Print User Info 62

For internal use only not for redistribution or sale

Introduction

This Document is created as reference material for MongoDB DBA Course. Course focuses primarily on understanding NoSQL Database concepts, MongoDB Architecture, CRUD Operations, MongoDB Administration, Performance Tuning and Replication and Sharding.

About Trainer

Mohammed Abdul Sami is an experienced DBA & System Administrator having more than 20 years of experience in managing Database like MongoDB, PostgreSQL, MySQL etc and Windows / Unix / Linux Servers. Since year 2015 he is working as freelance consultant and trainer.

Also Author is a frequent faculty at Oracle University for MySQL, Solaris, ZFS Storage Admin etc courses.

For internal use only not for redistribution or sale

Introduction

NoSQL Database

NoSQL is an approach to database design that can accommodate a wide variety of data models, including key-value, document, columnar and graph formats. NoSQL, which stand for "not only SQL," is an alternative to traditional relational databases in which data is placed in tables and data schema is carefully designed before the database is built. NoSQL databases are especially useful for working with large sets of distributed data.

Evolution of NoSQL

The evolution of NoSQL started in 1990's with the development and release of berkeleyDB. Developed by University of California, Berkeley, influenced the development of NoSQL which supported certain applications with specific storage needs.

NoSQL database Classification

NoSQL database is been classified in the following ways.

Key-value stores

Key-value stores, or key-value databases, implement a simple data model that pairs a unique key with an associated value. Because this model is simple, it can lead to the development of key-value databases, which are extremely performant and highly scalable for session management and caching in web applications.

Examples

Aerospike, Berkeley DB, MemchacheDB, Redis and Riak.

Document databases

Document databases, also called document stores, store semi-structured data and descriptions of that data in document format. They allow developers to create and update programs without needing to reference master schema. Use of document databases has increased along with use of JavaScript and the JavaScript Object Notation (JSON)

Examples

Couchbase Server, CouchDB, DocumentDB, MarkLogic and MongoDB.

Wide-column stores

Wide-column stores organize data tables as columns instead of as rows. Wide-column stores can be found both in SQL and NoSQL databases. Wide-column stores can query large data volumes faster than conventional relational databases.

Examples

Google BigTable, Cassandra and HBase.

Graph stores

Graph data stores organize data as nodes, which are like records in a relational database, and edges, which represent connections between nodes. Because the graph system stores the relationship between nodes, it can support richer representations of data relationships.

Examples

AllegroGraph, IBM Graph, Neo4j and Titan

by: Mohammed Abdul Sami

For internal use only not for redistribution or sale

NoSQL classification and Adherence by Vendors

The basic NoSQL database classifications are only guides. Over time, vendors have mixed and matched elements from different NoSQL database family trees to achieve more generally useful systems. That evolution is seen, for example, in MarkLogic, which has added a graph store and other elements to its original document databases. Couchbase Server supports both key-value and document approaches. Cassandra has combined key-value elements with a wide-column store and a graph database. Sometimes NoSQL elements are mixed with SQL elements, creating a variety of databases that are referred to as multimodel databases.

NoSQL vs RDBMS

SQL	NOSQL
RELATIONAL DATABASE MANAGEMENT SYSTEM (RDBMS)	Non-relational or distributed database system.
These databases have fixed or static or predefined schema	They have have dynamic schema
These databases are not suited for hierarchical data storage.	These databases are best suited for hierarchical data storage.
These databases are best suited for complex queries	These databases are not so good for complex queries
Verticlly Scalable	Horizontally scalable

CAP Theorem

The CAP Theorem is a fundamental theorem in distributed database systems that states any distributed system can have at most two of the following three properties.

- Consistency
- Availability
- Partition tolerance

Lets Understand CAP Theorem

The CAP theorem states that a distributed system cannot simultaneously be consistent, available, and partition tolerant.

Distributed databases

Distributed databases are usually non-relational databases that enable a quick access to data over a large number of nodes.

Consistency

Any read operation that begins after a write operation completes must return the latest value.

For internal use only not for redistribution or sale

In a consistent system, once a client writes a value to any server and gets a response, it expects to get that value (or a fresher value) back from any server it reads from.

Availability

Every request receives a (non-error) response – without the guarantee that it contains the most recent write

Partition Tolerance

The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes

PACELC theorem

PACELC theorem is an extension to the CAP theorem. It states that in case of network partitioning (P) in a distributed computer system, one has to choose between availability (A) and consistency (C) (as per the CAP theorem), but else (E), even when the system is running normally in the absence of partitions, one has to choose between latency (L) and consistency (C)

The purpose of PACELC is to address the oversight in CAP theorem pertaining to consistency/latency tradeoff of replicated systems as it is present at all times during system operation.

Markup Languages

What is JSON?

History

Douglas Crockford is typically credited with discovering the format because he was the first to popularize and specify its use around the year 2001.

Crockford, was developing an AJAX web application framework. However, needed to find a way to enable real-time two-way browser-to-server communication without relying on Flash plugins or Java applets. It was this need that served as the impetus behind the discovery of JSON. As JSON is used to transfer data between browser and server.

JSON consists of nothing more than commas, curly braces, square brackets, and data, it can be easily parsed into an array or object by any programming language.

JSON Data Structures

JSON data is formatted into two data structures that are used universally in all modern programming languages:

For internal use only not for redistribution or sale

- A JSON array is a list of values.

- A JSON object is a collection of name-value pairs.

JSON Arrays:

```
[ "red", "green", "blue", 7 ]
```

JSON Object (Document)

```
{
  name: "Sami"
  Occupation: "Trainer"
  Location: "Hyderabad"
}
```

Complex JSON objects:

```
{
  name: "Sami"
  Occupation: "Trainer"
  Location: "Hyderabad"
  Canteach: ["mongoDB", "MySQL", "PosgreSQL", "Linux"]
  LanguageFluency: {
      English:  "Very Good"
      Hindi: "Very Good"
      Telugu: "Very Good"
      Tamil: "Can Understand"
  }
}
```

BSON

BSON is a computer data interchange format. The name "BSON" is based on the term JSON and stands for "Binary JSON". It is a binary form for representing simple or complex data structures including associative arrays (also known as name-value pairs), integer indexed arrays, and a suite of fundamental scalar types. BSON originated in 2009 at MongoDB.

MongoDB represents JSON documents in binary-encoded format called BSON behind the scenes. BSON extends the JSON model to provide additional data types, ordered fields, and to be efficient for encoding and decoding within different languages

For internal use only not for redistribution or sale

What is YAML?

Yet Another Markup Language (YAML)

Is a human-readable data-serialization language. It is commonly used for configuration files, but could be used in many applications where data is being stored or transmitted. YAML targets many of the same communications applications as XML but has a minimal syntax. It uses both Python-style indentation to indicate nesting, and a more compact format that uses [] for lists and {} for maps making YAML a superset of JSON.

```
employee
    name: "Sami"
    Occupation: "Trainer"
    Location: "Hyderabad"
    Canteach:
        - mongoDB
        - MySQL
        - PosgreSQL
        - Linux
   LanguageFluency:
        English:  "Very Good"
        Hindi: "Very Good"
        Telugu: "Very Good"
       Tamil: "Can Understand"
```

Introduction to MongoDB

MongoDB is a document oriented database server developed in the C++ programming language. The word Mongo is derived from Humongous.

History

The initial development of MongoDB began in 2007 when the company was building a cloud platform.

MongoDB was developed by a NewYork based organization named 10gen which is now known as MongoDB Inc. It was initially developed as a PAAS (Platform As A Service). Later in 2009, it is introduced in the market as an open source database server that was maintained and supported by MongoDB Inc.

The first ready production of MongoDB has been considered from version 1.4 which was released in March 2010.

For internal use only not for redistribution or sale

Features of MongoDB

- Support adhoc queries

  In MongoDB, you can search by field, range query and it also supports regular expression searches.

- Indexing

  You can index any field in a document.

- Replication

  MongoDB supports Master Slave replication.

  A master can perform Reads and Writes and a Slave copies data from the master and can only be used for reads or back up (not writes)

- Duplication of data

  MongoDB can run over multiple servers. The data is duplicated to keep the system up and also keep its running condition in case of hardware failure.

- Load balancing

  It has an automatic load balancing configuration because of data placed in shards.

- Supports mapreduce and aggregation tools.

MongoDB Architecture

Client Connection Protocols:

The MongoDB Wire Protocol is a simple socket-based, request-response style protocol. Clients communicate with the database server through a regular TCP/IP socket.

- TCP/IP Socket

  Clients should connect to the database with a regular TCP/IP socket.

- Port

  The default port number for mongod and mongos instances is 27017.

- Default Clients

For internal use only not for redistribution or sale

The mongo shell is an interactive JavaScript interface to MongoDB. You can use the mongo shell to query and update data as well as perform administrative operations.

MongoDB Compass is the GUI client for MongoDB.

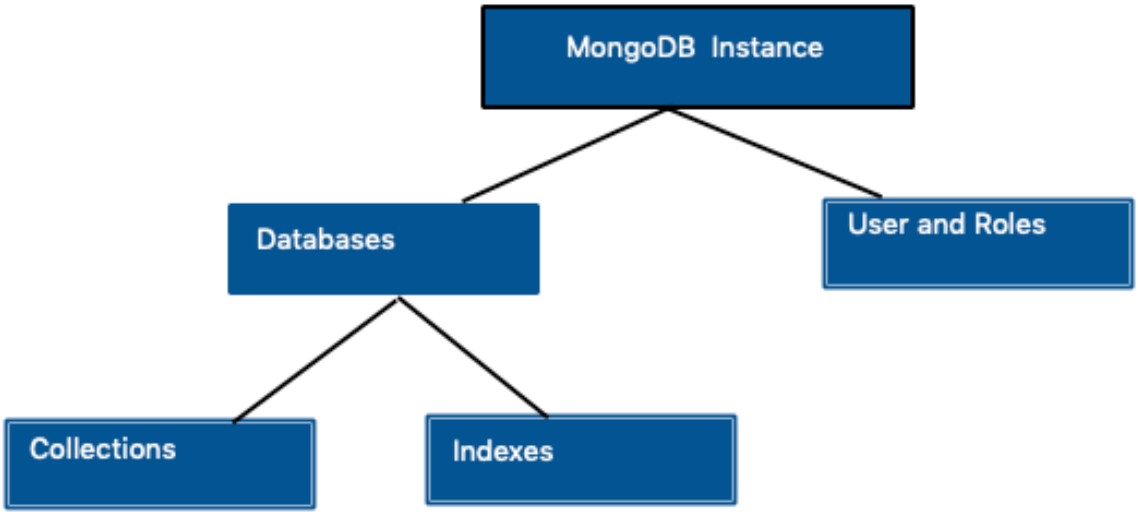
- Security

  MongoDB provides security by implementing user and roles. Security can be disabled.

- MongoDB Query Language

  MongoDB uses its own Query Language which is similar to Javascript and does not uses standard SQL.

Object Hierarchy



A MongoDB instance consisted of 2 high level objects called as DATABASES and USER/ROLES.

DATABASE in turn may have one or more COLLECTIONS and INDEXES which actually stores data.

MONGO DB Vs Traditional RDBMS

ROW	DOCUMENT
TABLE	COLLECTION

For internal use only not for redistribution or sale

COLUMN	PROPERTY or Field
One –Many Mapping	Enbbeded-document's

Data Types

MongoDB supports many datatypes. Some of them are −

String

This is the most commonly used datatype to store the data. String in MongoDB must be UTF-8 valid.

Integer

This type is used to store a numerical value. Integer can be 32 bit or 64 bit depending upon your server.

Boolean

This type is used to store a boolean (true/ false) value.

Double

This type is used to store floating point values.

Min/ Max keys

This type is used to compare a value against the lowest and highest BSON elements.

Arrays

This type is used to store arrays or list or multiple values into one key.

Timestamp

This can be handy for recording when a document has been modified or added.

Object

This datatype is used for embedded documents.

Null

This type is used to store a Null value.

Symbol

For internal use only not for redistribution or sale

This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.

Date

This datatype is used to store the current date or time in UNIX time format. You can specify your own date time by creating object of Date and passing day, month, year into it.

Object ID

This datatype is used to store the document's ID.

Binary data

This datatype is used to store binary data.

Code

This datatype is used to store JavaScript code into the document.

Regular expression

This datatype is used to store regular expression.

Storage

Storage Engines

The storage engine is the component of the database that is responsible for managing how data is stored, both in memory and on disk. MongoDB supports multiple storage engines, as different engines perform better for specific workloads. Choosing the appropriate storage engine for your use case can significantly improve the performance of your applications.

WiredTiger Storage Engine (Default)

WiredTiger is the default storage engine starting in MongoDB 3.2. It is well-suited for most workloads and is recommended for new deployments. WiredTiger provides a document-level concurrency model, checkpointing, and compression, among other features.

In MongoDB Enterprise, WiredTiger also supports Encryption at Rest. See Encrypted Storage Engine.

Document Level Concurrency

WiredTiger uses document-level concurrency control for write operations. As a result, multiple clients can modify different documents of a collection at the same time.

Uses Only Intent Lock

Any Conflict with two operations causes write conflict for one of the operation. MongoDB retries that operation.

For internal use only not for redistribution or sale

Global Lock

Operations involving multiple databases incur instance level lock (global lock).

Checkpointing

The checkpoint ensures that the data files are consistent up to and including the last checkpoint.

Checkpoint is performed every 60 seconds.

if MongoDB terminates or encounters an error while writing a new checkpoint, upon restart, MongoDB can recover from the last valid checkpoint.

Journal

WiredTiger uses a write-ahead log (i.e. journal) in combination with checkpoints to ensure data durability.

The WiredTiger journal persists all data modifications between checkpoints. If MongoDB exits between checkpoints, it uses the journal to replay all data modified since the last checkpoint.

Journaling Process

With journaling, WiredTiger creates one journal record for each client initiated write operation. The journal record includes any internal write operations caused by the initial write. For example, an update to a document in a collection may result in modifications to the indexes; WiredTiger creates a single journal record that includes both the update operation and its associated index modifications.

MongoDB configures WiredTiger to use in-memory buffering for storing the journal records. Threads coordinate to allocate and copy into their portion of the buffer. All journal records up to 128 kB are buffered.

WiredTiger syncs the buffered journal records to disk according to the following intervals or conditions:

MongoDB syncs the buffered journal data to disk every 50 milliseconds (Starting in MongoDB 3.2)

If the write operation includes a write concern of j: true, WiredTiger forces a sync of the WiredTiger journal files.

Because MongoDB uses a journal file size limit of 100 MB, WiredTiger creates a new journal file approximately every 100 MB of data. When WiredTiger creates a new journal file, WiredTiger syncs the previous journal file.

WiredTigerLog.<sequence> where <sequence> is a zero-padded number starting from 0000000001.

For internal use only not for redistribution or sale

Disabling Journaling

Wired Tiger storage engine, if journaling is disabled server does not guarantee the recovery of data after last checkpoint. In event of a crash data recovery is possible till last checkpoint.

Compression

With WiredTiger, MongoDB supports compression for all collections and indexes. Compression minimizes storage use at the expense of additional CPU.

Memory Use

With WiredTiger, MongoDB utilizes both the WiredTiger internal cache and the filesystem cache.

Starting in MongoDB 3.4, the default WiredTiger internal cache size is the larger of either:

- **50% of (RAM - 1 GB), or**
- **256 MB.**

MongoDB automatically uses all free memory that is not used by the WiredTiger cache or by other processes.

In-Memory Storage Engine

In-Memory Storage Engine is available in MongoDB Enterprise. Rather than storing documents on-disk, it retains them in-memory for more predictable data latencies.

Durability

The in-memory storage engine is non-persistent and does not write data to a persistent storage.

MMAPv1 Storage Engine (Deprecated as of MongoDB 4.0)

MMAPv1 is the default storage engine for MongoDB versions 3.0 and earlier. MMAPv1 is MongoDB's original storage engine based on memory mapped files. It excels at workloads with high volume inserts, reads, and in-place updates.

MongoDB Log

MongoDB log is system wide log which captures error, warning messages across the MongoDB.

By default mongoDB log is stored in /var/log directory on linux system can be changed to any location where process user has write permission.

For internal use only not for redistribution or sale

MongoDB Data Models

Data model design

- Embedded Data Models
- Normalized Data Models

Embedded Data Models

With MongoDB, you may embed related data in a single structure or document. These schema are generally known as “denormalized” models, and take advantage of MongoDB’s rich documents.

Example:

```
{
      _id: someid,
      name: "scott",
      Adresses: {
                Street: "10, oak street",
                State: "FL"
       }
}
```

In general, embedding provides better performance for read operations, as well as the ability to request and retrieve related data in a single database operation. Embedded data models make it possible to update related data in a single atomic write operation.

To access data within embedded documents, use dot notation

Example:

```
"Adresses.street" : "10, oak street"
```

Normalized Data Models

Normalized data models describe relationships using references between documents.

When to use normalized data models:

- When embedding would result in duplication of data but would not provide sufficient read performance advantages to outweigh the implications of the duplication.
- To represent more complex many-to-many relationships.
- To model large hierarchical data sets.

For internal use only not for redistribution or sale

- To join collections, MongoDB provides the aggregation stages:

  $lookup (Available starting in MongoDB 3.2)

  $graphLookup (Available starting in MongoDB 3.4)

Operational Factors and Data Models

Modeling application data for MongoDB should consider various operational factors that impact the performance of MongoDB.

Atomicity

In MongoDB, a write operation is atomic on the level of a single document, even if the operation modifies multiple embedded documents within a single document.

When a single write operation modifies multiple documents, the modification of each document is atomic, but the operation as a whole is not atomic.

Data Model Relationships

One-to-One Relationships with Embedded Documents

Consider the following normalized data

```
{
   _id: "joe",
   name: "Joe Bookreader"
}



{
   patron_id: "joe",
   street: "123 Fake Street",
   city: "Faketon",
   state: "MA",
   zip: "12345"
}
```

If the address data is frequently retrieved with the name information, then with referencing, your application needs to issue multiple queries to resolve the reference. The better data model would be to embed the address data in the patron data, as in the following document

```
{
   _id: "joe",
```


For internal use only not for redistribution or sale

```
   name: "Joe Bookreader",
   address: {
              street: "123 Fake Street",
              city: "Faketon",
              state: "MA",
              zip: "12345"
            }
}
```

Now our queries will be more efficient as with single read can retrieve all the needed information.

One-to-Many Relationships with Embedded Documents

Consider an example of patron data having multiple addresses

```
{
   _id: "joe",
   name: "Joe Bookreader"
}
```

```
{
   patron_id: "joe",
   street: "123 Fake Street",
   city: "Faketon",
   state: "MA",
   zip: "12345"
}
```

```
{
   patron_id: "joe",
   street: "1 Some Other Street",
   city: "Boston",
   state: "MA",
   zip: "67890"
}
```

If your application frequently retrieves the address data with the name information, then your application needs to issue multiple queries to resolve the references. A more optimal schema would be to embed the address data entities in the patron data, as in the following document:

For internal use only not for redistribution or sale

```
{
   _id: "joe",
   name: "Joe Bookreader",
   addresses: [
                {
                  street: "123 Fake Street",
                  city: "Faketon",
                  state: "MA",
                  zip: "12345"
                },
                {
                  street: "1 Some Other Street",
                  city: "Boston",
                  state: "MA",
                  zip: "12345"
                }
              ]
 }
```

One-to-Many Relationships with Document References

A data model that uses references between documents to describe one-to-many relationships between connected data.

Consider the following embedded document:

```
{
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",

   publisher: {

              name: "O'Reilly Media",

              founded: 1980,

              location: "CA"
```


For internal use only not for redistribution or sale

```
            }

}


{
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",

   publisher: {

               name: "O'Reilly Media",

               founded: 1980,

               location: "CA"

            }

}
```

Embedding the publisher document inside the book document would lead to repetition of the publisher data.

To avoid repetition of the publisher data, use references and keep the publisher information in a separate collection from the book collection.

```
{
   _id: "oreilly",
   name: "O'Reilly Media",
   founded: 1980,
   location: "CA"
}


{
```


For internal use only not for redistribution or sale

```
   _id: 123456789,
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher_id: "oreilly"
}


{
   _id: 234567890,
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher_id: "oreilly"
}
```


For internal use only not for redistribution or sale

Installation

For internal use only not for redistribution or sale

MongoDB Editions

MongoDB Community Server

The MongoDB Community Edition is free and available for Windows, Linux, and OS X.

MongoDB Enterprise Server

MongoDB Enterprise Server is the commercial edition of MongoDB, available as part of the MongoDB Enterprise Advanced subscription.

MongoDB Atlas

MongoDB is also available as an on-demand fully managed service. MongoDB Atlas runs on AWS, Microsoft Azure, and Google Cloud Platform

Supported platforms

MongoDB Community Edition installation includes:

Platform	Community Edition	Enterprise Edition
Linux	Install MongoDB Community Edition on Red Hat or CentOS Install MongoDB Community Edition on Ubuntu Install MongoDB Community Edition on Debian Install MongoDB Community Edition on SUSE Install MongoDB Community Edition on Amazon Linux	Install MongoDB Enterprise Edition on Red Hat or CentOS Install MongoDB Enterprise Edition on Ubuntu Install MongoDB Enterprise Edition on Debian Install MongoDB Enterprise Edition on SUSE Install MongoDB Enterprise Edition on Amazon Linux
macOS	Install MongoDB Community Edition on macOS	Install MongoDB Enterprise on macOS
Windows	Install MongoDB Community Edition on Windows	Install MongoDB Enterprise Edition on Windows

For internal use only not for redistribution or sale

Docker		Install MongoDB Enterprise with Docker

Note: MongoDB 3.6 and higher version does not support 32-bit platforms.

Installing Community Edition

Pre Requisites

MongoDB requires the following pre requisites on RHEL

- **libcurl**
- **lib openssl**
- **xz-libs**

Install Pre Requisites using following command

```
# yum install libcurl openssl xz-libs
```

Installing on RHEL using RPMs

Download MongoDB Community Server rpm from mongodb website

https://repo.mongodb.org/yum/redhat/7/mongodb-org/4.2/x86_64/RPMS/mongodb-org-server-4.2.14-1.el7.x86_64.rpm

Install the downloaded rpm

```
# rpm -ivh mongodb-org-server-4.2.14-1.el7.x86_64.rpm
```

Directory Paths

By default, MongoDB runs using the mongod user account and uses the following default directories:

/var/lib/mongo (the data directory)

/var/log/mongodb (the log directory)

Installing on RHEL using tgz

Download MongoDB Community Server tar ball from Mongodb website

https://fastdl.mongodb.org/linux/mongodb-linux-x86_64-rhel70-4.2.14.tgz

Create a service user for mongodb

For internal use only not for redistribution or sale

MongoDB uses **mongod** as service user if the user does not create it using following command.

```
# useradd mongod
```

Extract Files and copy them to /usr directory

The MongoDB binaries are in the bin/ directory of the tarball. You can use following commands

```
# tar -zxvf mongodb-linux-*-4.4.6.tgz
# cp /path/to/the/mongodb-directory/bin/* /usr/local/bin/
```

- Binaries are always owned by linux **root** user
- Binaries can be saved in any directory on system but make sure that directory is included in PATH variable.

Create a Data Directory and Log Directory

Create a data directory and log directory for mongodb. By default mongoDB uses following directories incase non default directories are used appropriate changes are required in configuration file.

```
mkdir -p /var/lib/mongo
mkdir -p /var/log/mongodb
```

Once the directories are created set the ownership to service user

```
sudo chown -R mongod:mongod <directory>
```

These directories can be created anywhere on system but make sure about ownership

Managing MongoDB Services

Managing Services with systemctl command

```
systemctl start  mongod
systemctl stop mongod
systemctl status mongod
```

Service Management for tar.gz Installation

Staring service

```
mongod  --dbpath  /ddata/directory
or
mongod --dbpath /data/directory  --logpath /log/file/path  --fork
```


For internal use only not for redistribution or sale

```
or
mongod -f /path/to/configuration/file
```

Shutting Down

```
> db.shutdownServer()
> db.shutdownServer({timeoutSecs: 60})
```

or Find the process id of **mongod** and use linux kill command

```
# ps -eaf| grep  mongod
# kill  <pid>
```

Connecting to MongoDB server using mongo client

MongoDB Default TCP/IP Port

MongoDB listens on 27001 TCP/IP Port by default

mongosh Client

The default MongoDB command line client is **mongosh** which can connect to mongoDB server. mongo client fully support **JavaScript**

```
$ mongosh
```

compass client

COMPASS is a GUI client provided by MongoDB which improves productivity of users

Compass can be downloaded from following link

https://downloads.mongodb.com/compass/mongodb-compass-1.26.1-win32-x64.exe

Default installation of MongoDB does not enable security which eliminates the need to provide credentials when connecting however when security is enabled proper credentials needs to provided while making a connection to MongoDB server.

Database and Collections

Databases in MongoDB contains collections which in turn stores the data. Basically Databases help organize data.

For internal use only not for redistribution or sale

System Databases

- Admin

  Security & Privilege information is stored in the collections of Admin Database

- Local:

  In a Replication setup details of local instance and other instance-specific data is stored. This database is invisible to replication and every node has its own Local Database.

- Config:

  This database is used by Sharding to store session data. Normal user or administrator or not allowed to write in this database but for the purpose of diagnosing issues with sharding a single document can be inserted.

Database Commands

Listing all Databases

The **`show dns`** method can be used to list all databases

```
> show dbs
```

Changing to a Database

The **`use`** method can be used to switch databases

```
> use  <databasename>
```

What is Current Database

`db` command can be used to find out current database

```
> db
```

Creating a New Database

A new database is created with the **`use`** method along with new database name

```
> use  mynewdb
```

However to fully create a database we need to create at least one collection in it and insert at least one document in that collection.

Dropping a Database

`db.dropDatabase()` method can be used drop the current database

For internal use only not for redistribution or sale

```
> use <mydb>
> db.dropDatabase()
```

Collections

List all Collections

```
> show collections
```

Create Collection

Good thing in MongoDB is there is no need to explicitly create collection as they are created automatically when we insert the first document in it. However there is **`db.createCollection()`** method to create a collection.

Example:

```
> db.createCollection("clients")
```

Drop a Collection

Switch the database from which you want to drop the collection then issue the db.collection_name.drop() method to the collection named collection_name.

```
> use  <mydb>
> db.<collection name>.drop()
```

For example, from test database I want to drop collection named addresses

```
> use test
> db.addresses.drop()
```


For internal use only not for redistribution or sale

CRUD Operations

by: Mohammed Abdul Sami

For internal use only not for redistribution or sale

CRUD stands for Create, Read, Update and Delete basically they are equivalent to DML operations in RDBMS.

MongoDB Operators

Comparison Operators:

Name	Description
$eq	Matches values that are equal to a specified value.
$gt	Matches values that are greater than a specified value.
$gte	Matches values that are greater than or equal to a specified value.
$in	Matches any of the values specified in an array.
$lt	Matches values that are less than a specified value.
$lte	Matches values that are less than or equal to a specified value.
$ne	Matches all values that are not equal to a specified value.
$nin	Matches none of the values specified in an array.
$regex	Matches values as per regular expression given

Logical Operators:

Name	Description

For internal use only not for redistribution or sale

`$and`	Joins query clauses with a logical AND returns all documents that match the conditions of both clauses.
`$not`	Inverts the effect of a query expression and returns documents that do *not* match the query expression.
`$nor`	Joins query clauses with a logical NOR returns all documents that fail to match both clauses.
`$or`	Joins query clauses with a logical `OR` returns all documents that match the conditions of either clause.

Element Operators

Name	Description
`$exists`	Matches documents that have the specified field.
`$type`	Selects documents if a field is of the specified type.

The Crud Operations

insert Method

Inserts a document or documents into a collection. There are 2 methods for inserting the documents

```
db.collection.insertOne( { } )
db.collection.insertMany( { } )
```


For internal use only not for redistribution or sale

Syntax:

```
db.collection.insertOne([
   {doc1},{doc2},....],
   {
     writeConcern: <document>,
     ordered: <boolean>
   }
)
```

Examples:

```
db.products.insertOne( { item: "card", qty: 15 } )
```

```
db.products.insertMany(
   [
     { _id: 11, item: "pencil", qty: 50, type: "no.2" },
     { item: "pen", qty: 20 },
     { item: "eraser", qty: 25 }
   ]
)
```

Unordered Inserts:

```
db.products.insertMany(
   [
     { _id: 20, item: "lamp", qty: 50, type: "desk" },
     { _id: 21, item: "lamp", qty: 20, type: "floor" },
     { _id: 22, item: "bulk", qty: 100 }
   ],
   { ordered: false }
)
```

Inserts with Write Concerns:

```
db.products.insertOne(
    { item: "envelopes", qty : 100, type: "Clasp" },
    { writeConcern: { w: "majority", wtimeout: 5000 } }
)
```


For internal use only not for redistribution or sale

insertMany Method

Inserts multiple documents into the collection

```
db.collection.insertMany(
   [ {<document 1>} , {<document 2>}, ... ],
   {
      writeConcern: <document>,
      ordered: <boolean>
   }
)
```

find Method

Selects documents in a collection or view and returns the selected documents.

Syntax:

```
db.collection.find({query}, {projection})
```

query: is a condition to match the documents

Projection: Selection of fields to return. Projection cannot contain both exclude and include specification except for _id field

Query Examples:

```
db.bios.find( { qty: 23 } )
db.bios.find( { "name.last": "Hopper" } )
```

Note: To access fields in an embedded document, use dot notation (`"<embedded document>.<field>"`).

Using $in Operator

```
db.employees.find(
   { _id: { $in: [ 1001, 1005) ] } }
)
```

Using $in Operator on Array to match any of the element

```
db.employees.find(
   { canteach: { $in: [ "mongodb", "MySQL") ] } }
)
```


For internal use only not for redistribution or sale

Using Regex with find method

```
db.employees.find(
   { name: { $regex: /^N/ } }
)
```

Using comparison Operators

```
db.employees.find( { doj: { $gt: new Date('2015-01-01') } } )
db.employees.find( { salary: { $gt: 10000 } } )
db.employees.find( { salary: { $lt: 10000 } } )
```

Range Queries:

```
db.employees.find( { salary: { $gte: 10000, $lt: 20000 } } )
db.employees.find( { doj: { $gt: new Date('2015-01-01'), $lt: new
Date('2017-12-31') } } )
```

Queries on Multiple fields

```
db.employees.find( {
   doj: { $gt: new Date('1920-01-01') },
   salary: { $gt : 10000}
} )
```

Querying for documents without a particular field

```
db.employees.find( {
   doj: { $exists: false }
} )
db.employees.find( {
   doj: null
} )
```

Querying an Array

```
db.employees.find( { canteach: { $all: [ "mongodb", "mysql" ] } } )

db.employees.find(
   { canteach: { $in: [ "mongodb", "MySQL") ] } }
)
```


For internal use only not for redistribution or sale

```
db.employees.find( { canteach: { $size: 2} } )
```

$elemmatch : The **$elemMatch** operator matches documents that contain an array field with at least one element that matches all the specified query criteria

```
Consider the following collection
{ _id: 1, results: [ 82, 85, 88 ] }
{ _id: 2, results: [ 75, 88, 89 ] }


db.scores.find(
   { results: { $elemMatch: { $gte: 80, $lt: 85 } } }
)
```

Specify the Fields to Return (projection)

```
db.employees.find( {}, {_id:0,  name:1, salary:1 } } )
```

Sorting Result

Sorting the Result (sort() method)

```
db.employees.find().sort({name: 1})
```

Limiting output

Limit method limit()

```
db.employees.find().limit(5)
```

Skip method skip()

```
db.employees.find().skip(5)
```

Updating Data

Modifies an existing document or documents in a collection. The method can modify specific fields of an existing document or documents or replace an existing document entirely, depending on the update parameter.

Data from the collections can be modified by the following 2 methods

```
db.collection.updateOne()
db.collection.updateMany()
```

UpdateOne() Method

```
db.collection.updateOne( {query}, {update}, {options})
```


For internal use only not for redistribution or sale

The first document matching the query will be updated

Syntax:

```
db.collection.update(
   <query>,
   <update>,
   {
     upsert: <boolean>,
     multi: <boolean>,
     writeConcern: <document>,

   }
)
```

Upsert Behavior

If upsert is true and no document matches the query criteria, update() inserts a single document. The update creates the new document.

If upsert is true and there are documents that match the query criteria, update() performs an update.

```
db.people.update(
   { name: "Andy" },
   {
      name: "Andy",
      rating: 1,
      score: 1
   },
   { upsert: true }
)
```

Updating Multiple Documents

If multi is set to true, the updateMany() method updates all documents that meet the <query> criteria.

```
db.books.updateMany(
   { stock: { $lte: 10 } },
   { $set: { reorder: true } },
   { multi: true }
)
```


For internal use only not for redistribution or sale

With write concern

```
db.books.update(
   { stock: { $lte: 10 } },
   { $set: { reorder: true } },
   {
     multi: true,
     writeConcern: { w: "majority", wtimeout: 5000 }
   }
)
```

Changing Data

Changes to documents can be done using any of the `$set` or `$inc` Operators

Example of `$set`:

```
db.books.updateOne(
   { _id: 1 },
   {
     $set: {
       item: "ABC123",
       "info.publisher": "2222",
       tags: [ "software" ],
       "ratings.1": { by: "xyz", rating: 3 }
     }
   }
)
```

Example of $inc operator:

```
db.books.updateOne(
   { _id: 1 },
   { $inc: {stock:5}
 )
```

Remove Fields

Remove Fields from a Document

For internal use only not for redistribution or sale

The following operation uses the $unset operator to remove the field

```
db.employees.updateOne( { _id: 1 }, { $unset: { salary: 1 } } )
```

Deleting Data

Delete Method

delete method deletes the documents matching given query. There are 2 variations namely

```
db.collection.deletemany()
db.collection.deleteOne().
```

```
db.collection.deleteMany(
   {<filter>},
   {
      writeConcern: <document>,
      collation: <document>
   }
)
db.collection.deleteOne(
   {<filter>},
   {
      writeConcern: <document>,
      collation: <document>,
      hint: <Index hint>
  }
)
```

Advance Collection Creation

MongoDB is schema less, means there is no fixed schema or field definitions for collections however MongoDB allow to specify certain validations such as size limit for collection or required fields.

Creating capped collection

Creating capped collection requires us to provide SIZE and MAX values.

SIZE: Maximum size of the collections. When Collection reaches this size then the oldest documents are deleted to make room for new documents. MongoDB ensures that the collection does not grow above the mention size. Mentioning SIZE is must for capped collections.

For internal use only not for redistribution or sale

MAX: specifies maximum number of document a collection can have. SIZE takes the precedence when SIZE limit is reached before reaching MAX.

Example:

```
db.createCollection("stock", { capped : true, size : 5242880, max :
5000 } )
```

The above example creates a collection named stock with 5MB size cap.

Collection Validations

MongoDB does not enforce fixed schema requirements but it provides validations to declare required field list and field properties like datatype, input validations etc..

Required Fields

Creating a collections with required fields

```
db.createCollection( "employees", {
   validator: { $jsonSchema: {
      bsonType: "object",
      required: [ "ename", "job", "salary" ] }
    }
  }
)
```

A new collection named "employees" is created where every document required to have "ename", "job" and "salary" columns.

Input Validation

Schema validations allows to specify datatype and other input validations for documents in a collection. properties object which is embedded object within validator object facilitates input validations.

```
     properties: {
         field1 : {
            bsonType: "<type>",
             enum : [v1,v2,....],
            description: "<some description>"
         },
         field2: {
            bsonType: "<type>",
             minimum : <number>,
```


For internal use only not for redistribution or sale

```
            maximum : <number>,
            description: "<some descriptions>"
         }
      }
```

Example:

```
db.createCollection( "employee2", {
   validator: { $jsonSchema: {
      bsonType: "object",
      required: [ "ename", "job", "salary" ],
      properties: {
         ename : {
            bsonType: "string",
            description: "Name of the Employee"
         },
         job: {
            bsonType: "string",
               enum : ["Manager", "Analyst","Clerk","Salesman"],
            description: "Designation of Employee, Choose one of the
mentioned jobs"
         },
         salary: {
            bsonType: "decimal",
            minimum : 1000,
               maximum : 1000000,
          description: "Salary of Employee"
         },
         deptno: {
         bsonType: "int",
         description: "Department number of Employee"
         }
      }
      }
    }
  }
)
```

View Collection Validation Specification use following method.

```
db.getCollectionInfos()
```


For internal use only not for redistribution or sale

For internal use only not for redistribution or sale

Aggregation

by: Mohammed Abdul Sami

For internal use only not for redistribution or sale

The aggregation pipeline is a framework for data aggregation modeled on the concept of data processing pipelines. Documents enter a multi-stage pipeline that transforms the documents into aggregated results.

Syntax:

```
db.collection.aggregate([ {stage1}, {stage2}, {stage3},....])
```

An aggregation pipeline consists of one or more stages that process documents:

- Each stage performs an operation on the input documents. For example, a stage can filter documents, group documents, and calculate values.
- The documents that are output from a stage are passed to the next stage.
- An aggregation pipeline can return results for groups of documents. For example, return the total, average, maximum, and minimum values.

Aggregation stages

$match stage

Usage:

```
{ $match: { <query> } }
```

Example:

```
db.emp.aggregate([{$match: {job:"CLERK"}}])
```

In Aggregation **$match** stage can come at any position.

$group Stage

Group stage perform group level aggregation computations like sum, count, min, max etc. For group level computations it further needs accumulators as shown in table below.

Name	Description
$avg	Returns an average of numerical values. Ignores non-numeric values.
$first	Returns a value from the first document for each group. Order is only defined if the documents are in a defined order.

For internal use only not for redistribution or sale

$last	Returns a value from the last document for each group. Order is only defined if the documents are in a defined order.
$max	Returns the highest expression value for each group.
$min	Returns the lowest expression value for each group.
$stdDevPop	Returns the population standard deviation of the input values.
$stdDevSamp	Returns the sample standard deviation of the input values.
$sum	Returns a sum of numerical values. Ignores non-numeric values.

Grouping Examples:

Sum of Salary for each job title:

```
db.emp.aggregate([ {$group:{ _id:"$job", tot_sal:{$sum:"$sal"}}}])
```

Number of employees per job title:

```
db.emp.aggregate([ {$group:{ _id:"$job", No_OF_Emp:{$sum:1}}}])
```

Total No. of employees:

```
db.emp.aggregate([ {$group:{ _id:null, All_Emp_COUNT:{$sum:1}}}])
```

Lowest Salary:

```
db.emp.aggregate([ {$group:{ _id:null, All_Emp_COUNT:{$min: "$sal" }
}}])
```

Highest Salary:

For internal use only not for redistribution or sale

```
db.emp.aggregate([ {$group:{ _id:null,
All_Emp_COUNT:{$max:"$sal"}}}])
```

$bucket stage

Syntax:

```
{
  $bucket: {
      groupBy: <expression>,
      boundaries: [ <lowerbound1>, <lowerbound2>, ... ],
      default: <literal>,
      output: {
         <output1>: { <$accumulator expression> },
         ...
         <outputN>: { <$accumulator expression> }
      }
   }
}
```

Lets say that we want categorized products according to salePrice and count number of products within each price range.

Here the price range we are using is called as boundaries based on which grouping is done for salePrice field

Example:

```
db.emp.aggregate([
{$bucket:{groupBy:"$sal",boundaries:[0,1000,2000,5000],default:"Other
",output:{"count":{$sum:1}}}}])
```

Example of bucket with 2 output fields fist will count no. of products in each bucket and second will report what is the maximum salePrice in each bucket

```
db.emp.aggregate([
{$bucket:{groupBy:"$sal",boundaries:[0,1000,2000,3000,4000,5000],defa
ult:"Other",output:{"count":{$sum:1},"max":{$max:"$sal"}}}}])
```

$bucketAuto stage

Here it will automatically decides on the boundaries and produce result

For internal use only not for redistribution or sale

syntax:

```
{
  $bucketAuto: {
      groupBy: <expression>,
      buckets: <number>,
      output: {
         <output1>: { <$accumulator expression> },
         ...
      }
  }
}
```

Example:

```
db.emp.aggregate([
{$bucketAuto:{groupBy:"$sal",buckets:4,output:{"count":{$sum:1},"max"
:{$max:"$sal"}}}}])
```

$addFields stage

Adds new fields to documents. $addFields outputs documents that contain all existing fields from the input documents and newly added fields

Syntax:

```
{ $addFields: { <newField>: <expression>, ... } }
```

Lets add a new field called Annual Salary which is computed 12 times of sal.

```
db.emp.aggregate([ {$addFields:
{Annual_Salary:{$multiply:["$sal",12]}}}])
```

$project stage

Passes along the documents with the requested fields to the next stage in the pipeline. The specified fields can be existing fields from the input documents or newly computed fields.

Syntax:

```
{ $project: { <specification(s)> } }
```

Example:

```
db.emp.aggregate([ {$project: {_id:0,ename:1,job:1, sal:1}}])
```

$project and $addField Together

For internal use only not for redistribution or sale

```
db.emp.aggregate([ {$addFields: {Ann_Sal:{$multiply:["$sal",12]}}},
{$project: {_id:0,ename:1,job:1, sal:1, Ann_Sal:1}} ])
```

$lookup stage

Performs a left outer join to an un-sharded collection in the same database to filter in documents from the “joined” collection for processing.

Syntax:

```
{
   $lookup:
     {
       from: <collection to join>,
       localField: <field from the input documents>,
       foreignField: <field from the documents of the "from"
collection>,
       as: <output array field>
     }
}
```

For example if we have a dept collection with deptno, name and location filelds and an employees collection with a matching deptno filed we can match both the collections.

```
db.dept.aggregate([{ $lookup: {
from:  "emp",
localField: "_id",
foreignField: "deptno",
as:  "employee-details"
}
}
]
)
```


For internal use only not for redistribution or sale

MongoDB Security and User Administration

by: Mohammed Abdul Sami

For internal use only not for redistribution or sale

Authentication Mechanism

challenge-response

MONGODB-CR is a challenge-response mechanism that authenticates users through passwords.

This mechanism is deprecated in version 4.0 and in version 5.x or above this must not be used.

SCRAM

Salted Challenge Response Authentication Mechanism (SCRAM) is the default authentication mechanism for MongoDB. SCRAM is based on the IETF RFC 5802 standard that defines best practices for implementation of challenge-response mechanisms for authenticating users with passwords.

Using SCRAM, MongoDB verifies the supplied user credentials against the user's name, password and authentication database. The authentication database is the database where the user was created, and together with the user's name, serves to identify the user.

SCRAM Mechanisms

MongoDB supports the following SCRAM mechanisms:

- SCRAM-SHA-1 : Uses the SHA-1 hashing function
- SCRAM-SHA-256 : Uses the SHA-256 hashing function

When creating or updating a SCRAM user, you can indicate the specific SCRAM mechanism

x.509

MongoDB supports x.509 certificate authentication for client authentication and internal authentication of the members of replica sets and sharded clusters.

x.509 certificate authentication requires a secure TLS/SSL connection.

Authentication in Enterprise version of MongoDB

The Enterprise version of MongoDB supports following authentication methods.

- Kerberos Authentication
- LDAP Proxy Authentication

For internal use only not for redistribution or sale

Role-Based Access Control

MongoDB employs Role-Based Access Control (RBAC) to govern access to a MongoDB system. A user is granted one or more roles that determine the user's access to database resources and operations. Outside of role assignments, the user has no access to the system.

Resource

A resource can be a database, collection, set of collections, or the cluster. If the resource is the cluster, the affiliated actions affect the state of the system rather than a specific database or collection.

Privileges

A privilege consists of a specified resource and the actions permitted on the resource.

Roles

A role grants privileges to perform the specified actions on resource. Each privilege is either specified explicitly in the role or inherited from another role or both.

Inherited Privileges

A role can include one or more existing roles in its definition, in which case the role inherits all the privileges of the included roles.

A role can inherit privileges from other roles in its database. A role created on the admin database can inherit privileges from roles in any database.

Viewing Roles

MongoDB provides methods to view the built-in roles and view details of specific role.

Built-In Roles

Following code gets all the built0in roles.

```
db.runCommand(
    {
      rolesInfo: 1,
      showBuiltinRoles: true
    }
)
```

Viewing details of a particular role

For internal use only not for redistribution or sale

Following code shows details of the particular role.

```
db.runCommand(
    {
      rolesInfo: { role: "<rolename>", db: "<database name>" },
      showPrivileges: true
    }
)
```

Example:

For Example we need to know the details of the role developer then the following code fetches them.

```
db.runCommand(
    {
      rolesInfo: { role: "developer", db: "mydb" },
      showPrivileges: true
    }
)
```

Privileges (Actions)

Each role consists of certain privileges which determines what action user can perform. Following are the some of the common privileges (actions)

collStats	killCursors	convertToCapped	repairDatabase
convertToCapped	listIndexes	createCollection	storageDetails
createCollection	listCollections	createIndex	validate
dbHash	remove	dbStats	changeCustomData
dbStats	renameCollection SameDB	dropCollection	changePassword
dropCollection	update	dropDatabase	createRole

For internal use only not for redistribution or sale

createIndex	bypassDocument Validation	dropIndex	createUser
dropIndex	collMod	enableProfiler	dropRole
find	collStats	reIndex	dropUser
insert	compact	renameCollectionSameDB	grantRole

Built-In Roles

MongoDB provides built-in roles that provide set of privileges commonly needed in a database system.

MongoDB provides built-in roles at

- Server Level (Super User)
- Database level
- cluster level

Super User Role

`root` role Provides access to the operations and all the resources of the readWriteAnyDatabase, dbAdminAnyDatabase, userAdminAnyDatabase, clusterAdmin, r estore, and backup combined.

Database Level

Database Level roles are of 4 types

- Database User Roles
- Database Admin Roles

For internal use only not for redistribution or sale

- Backup & Restore Roles
- All-Database Roles

Database User Role

- read : Able read from collection
- readWrite : Able to read & write collections

Database Admin Roles

- dbAdmin :
- dbOwner : Owner of the databases
- userAdmin : User Admin Role within database

Backup & Restore Roles

- backup : Able to take backup of the database
- restore : Able to perform database restore from the backup

All-Database Roles

The following roles are available on the admin database and provide privileges which apply to all databases except local and config

- readAnyDatabase : Read from any database
- readWriteAnyDatabase : Read & write to any database
- userAdminAnyDatabase : User Admin any database
- dbAdminAnyDatabase : DB admin of any database

Cluster Level

- clusterAdmin :
- clusterMonitor
- hostManager

For internal use only not for redistribution or sale

User-Defined Roles

If the built-in roles cannot describe the desired set of privileges, you can create new roles.

MongoDB provides methods to create or modify the roles as needed.

The roles created for a particular databases applies to only that database whereas if the role created in Admin Database then it applies to all databases.

Creating User-Defined Role:

Create role syntax:

```
db.createRole( {
  role: "<name>",
  privileges: [
     { resource: { <resource> }, actions: [ "<action>", ... ] },
     ...
  ],
  roles: [
     { role: "<role>", db: "<database>" } | "<role>",
      ...
  ],
  authenticationRestrictions: [
    {
      clientSource: ["<IP>" | "<CIDR range>", ...],
      serverAddress: ["<IP>" | "<CIDR range>", ...]
    },
    ...
  ]
})
```

Example1: (Creating a role for mydb & mov dns)

```
use mydb
db.createRole(
   {
     role: "mycustrole",
     privileges: [

       { resource: { db: "mydb", collection: "coll1" }, actions: [
"find", "update", "insert", "remove" ] },
```

by: Mohammed Abdul Sami

For internal use only not for redistribution or sale

```
        { resource: { db: "mydb", collection: "coll2" }, actions: [
"update", "insert", "remove" ] }
      ],
      roles: [
        { role: "read", db: "admin" }
      ]
    },
    { w: "majority" , wtimeout: 5000 }
)
```

Example2: (Creating Cluster Admin role)

```
use admin
db.createRole(
    {
      role: "myClusterwideAdmin",
      privileges: [
        { resource: { cluster: true }, actions: [ "addShard" ] },
        { resource: { db: "config", collection: "" }, actions: [
"find", "update", "insert", "remove" ] },
        { resource: { db: "users", collection: "usersCollection" },
actions: [ "update", "insert", "remove" ] },
        { resource: { db: "", collection: "" }, actions: [ "find" ] }
      ],
      roles: [
        { role: "read", db: "admin" }
      ]
    },
    { w: "majority" , wtimeout: 5000 }
)
```

Dropping a User-Defined Role

Using db.dropRole() method a role can be dropped.

Syntax:

```
db.dropRole(rolename, writeConcern)
```


For internal use only not for redistribution or sale

Example:

```
use mydb
db.dropRole( "mycustrole", { w: "majority" } )
```

Dropping all User-Defined roles at once

```
db.dropAllRoles(writeConcern)
```

Example:

```
use mydb
db.dropAllRoles( { w: "majority" } )
```

Displaying Information on Roles:

```
db.getRole(rolename)
```

Example:

```
use mydb
db.getRole(mycustrole)
```

Example:

```
use mydb
db.getRoles()
```

Granting & Revoking Privileges to a role

Any time additional privileges can be granted or existing privileges can be revoked from the roles. A role can also be granted / revoked another role.

Granting Privileges:

```
db.grantPrivilegesToRole(
    "< rolename >",
    [
        { resource: { <resource> }, actions: [ "<action>", ... ] },
        ...
    ],
    { < writeConcern > }
)
db.grantRolesToRole( "<rolename>", [ <roles> ], { <writeConcern> } )
```

Revoking Privileges:

For internal use only not for redistribution or sale

```
db.revokePrivilegesFromRole(
    "<rolename>",
    [
        { resource: { <resource> }, actions: [ "<action>", ... ] },
        ...
    ],
    { <writeConcern> }
)
db.revokeRolesFromRole( "<rolename>", [ <roles> ], { <writeConcern> }
)
```

Updating Role:

Updating role replaces previous privileges with new ones. Syntax of update role same as db.createRole

Syntax:

```
db.updateRole(
    "<rolename>",
    {
      privileges:
          [
            { resource: { <resource> }, actions: [ "<action>", ... ]
},
            ...
          ],
      roles:
          [
            { role: "<role>", db: "<database>" } | "<role>",
            ...
          ],
      authenticationRestrictions:
          [
            {
              clientSource: ["<IP>" | "<CIDR range>", ...],
              serverAddress: ["<IP>", | "<CIDR range>", ...]
            },
            ...
          ]
    },
```


For internal use only not for redistribution or sale

```
    { <writeConcern> }
)
```

Enabling Security

Put the below line mongodb configuration file to enable security and authorization in mongoDB. Service restart is needed.

```
security:
  authorization: enabled
```

By default MongoDB does not include a built-in super user. Before or immediately after enabling security authorization it is highly recommended to create a super user.

Localhost Exception

The localhost exception allows a password less connection to MongoDB if the security is enabled without creating a super user account first. The very first user created must be a super user or atleast a user with privileges to create and manage other user.

Creating Super User

```
use admin
db.createUser({
  user: "admin",
  pwd: "pass12345",
  roles: [ {role: "root", db: "admin"} ]
})
```

Any user with `"root"` role on "admin" database is super user

User Management

Creating Users:

Create security officer user which can create or modify other users

```
db.createUser(
  { user: "security_officer",
    pwd: "h3ll0th3r3",
    roles: [ { db: "admin", role: "userAdmin" } ]
  }
)
```

Any user created associated with userAdmin role can manage other user/ create new users.

For internal use only not for redistribution or sale

Create database administrator:

```
db.createUser(
  { user: "dba",
    pwd: "c1lynd3rs",
    roles: [ { db: "admin", role: "dbAdmin" } ]
  }
)
```

Connecting To MongoDB Using User

Once user is created we can connect to mongo shell using the newly created user

```
mongo --authenticationDatabase <dbName> -u <userName> -p
```

Deleting a user

```
db.dropUser('<userName>')
```

Granting Roles to User

```
db.grantRolesToUser( "<username>", [ <roles> ], { <writeConcern> } )
```

Example:

```
use mydb
db.grantRolesToUser(
   "dba",
   [ "readWrite" , { role: "read", db: "mydb" } ],
   { w: "majority" , wtimeout: 4000 }
)
```

Revoking Roles from User

```
db.revokeRolesFromUser( "<username>", [ <roles> ], { <writeConcern> }
)
```

Example:

```
use mydb
db.revokeRolesFromUser( "dba", [ { role: "read", db: "mydb" },
"readWrite" ], { w: "majority" } )
```

Changing User Password

```
db.changeUserPassword("username","Password")
```


For internal use only not for redistribution or sale

Authenticate to Another Database within Shell

```
db.auth( <username>, <password> )
```

Print User Info

```
use mydb
db.getUser("dba")
db.getUsers()
```

Appendix 1

revokeRole

setAuthenticationRestriction

For internal use only not for redistribution or sale

viewRole

viewUser

addShard

appendOplogNote

applicationMessage

cleanupOrphaned

flushRouterConfig

listSessions (New in version 3.6)

listShards

removeShard

replSetConfigure

replSetGetConfig

replSetGetStatus

replSetStateChange

resync

enableSharding

moveChunk

splitChunk

splitVector

applicationMessage

closeAllDatabases

connPoolSync

cpuProfiler

flushRouterConfig

For internal use only not for redistribution or sale

fsync

invalidateUserCache

killAnyCursor (New in version 4.0)

killAnySession (New in version 3.6)

killop

logRotate

resync

setParameter

shutdown

touch

unlock

For internal use only not for redistribution or sale